



ReposVul: A Repository-Level High-Quality Vulnerability Dataset

Xinchen Wang*
Harbin Institute of Technology,
Shenzhen, China
200111115@stu.hit.edu.cn

Ruida Hu*
Harbin Institute of Technology,
Shenzhen, China
200111107@stu.hit.edu.cn

Cuiyun Gao*
Harbin Institute of Technology,
Shenzhen, China
gaocuiyun@hit.edu.cn

Xin-Cheng Wen
Harbin Institute of Technology,
Shenzhen, China
xiamenwxc@foxmail.com

Yujia Chen
Harbin Institute of Technology,
Shenzhen, China
yujiachen@stu.hit.edu.cn

Qing Liao
Harbin Institute of Technology,
Shenzhen, China
liaoqing@hit.edu.cn

ABSTRACT

Open-Source Software (OSS) vulnerabilities bring great challenges to the software security and pose potential risks to our society. Enormous efforts have been devoted into automated vulnerability detection, among which deep learning (DL)-based approaches have proven to be the most effective. However, the performance of the DL-based approaches generally relies on the quantity and quality of labeled data, and the current labeled data present the following limitations: (1) **Tangled Patches**: Developers may submit code changes unrelated to vulnerability fixes within patches, leading to tangled patches. (2) **Lacking Inter-procedural Vulnerabilities**: The existing vulnerability datasets typically contain function-level and file-level vulnerabilities, ignoring the relations between functions, thus rendering the approaches unable to detect the inter-procedural vulnerabilities. (3) **Outdated Patches**: The existing datasets usually contain outdated patches, which may bias the model during training.

To address the above limitations, in this paper, we propose an automated data collection framework and construct the first repository-level high-quality vulnerability dataset named **ReposVul**. The proposed framework mainly contains three modules: (1) A vulnerability untangling module, aiming at distinguishing vulnerability-fixing related code changes from tangled patches, in which the Large Language Models (LLMs) and static analysis tools are jointly employed. (2) A multi-granularity dependency extraction module, aiming at capturing the inter-procedural call relationships of vulnerabilities, in which we construct multiple-granularity information for each vulnerability patch, including repository-level, file-level, function-level, and line-level. (3) A trace-based filtering module, aiming at filtering the outdated patches, which leverages the file path trace-based filter and commit time trace-based filter to construct an up-to-date dataset.

* These authors contribute to the work equally and are co-first authors of the paper.
* Corresponding author. The author is also affiliated with Peng Cheng Laboratory and Guangdong Provincial Key Laboratory of Novel Security Intelligence Technologies.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ICSE-Companion '24, April 14–20, 2024, Lisbon, Portugal

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0502-1/24/04
<https://doi.org/10.1145/3639478.3647634>

The constructed repository-level ReposVul encompasses 6,134 CVE entries representing 236 CWE types across 1,491 projects and four programming languages. Thorough data analysis and manual checking demonstrate that ReposVul is high in quality and alleviates the problems of tangled and outdated patches in previous vulnerability datasets.

CCS CONCEPTS

• **Software and its engineering** → **Software defect analysis**.

KEYWORDS

Open-Source Software, Software Vulnerability Datasets, Data Quality

ACM Reference Format:

Xinchen Wang*, Ruida Hu*, Cuiyun Gao*, Xin-Cheng Wen, Yujia Chen, and Qing Liao. 2024. ReposVul: A Repository-Level High-Quality Vulnerability Dataset. In *2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion '24)*, April 14–20, 2024, Lisbon, Portugal. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3639478.3647634>

1 INTRODUCTION

In recent years, with the increasing size and complexity of Open-Source Software (OSS), the impact of OSS vulnerabilities has also amplified and can cause great losses to our society.

For example, Cisco discovered a security vulnerability in the WebUI, identified as CVE-2023-20198 [1] in 2023. This vulnerability allowed unauthorized remote attackers to gain elevated privileges. Currently, over 41,000 related devices have been compromised, resulting in great losses for enterprises. Identifying vulnerabilities in an accurate and timely manner is beneficial for mitigating the potential risks, and has gained intense attention from industry and academia. The existing vulnerability detection methods can be coarsely grouped into two categories: program analysis-based methods [2–5] and deep learning (DL)-based methods [6–9], among which DL-based methods have proven to be more effective. Despite the success of the DL-based methods, their performance tends to be limited by the trained vulnerability datasets. For example, Croft et al. [10] find that the widely-used vulnerability datasets such as Devign [11] and BigVul [12] contain noisy, incomplete and outdated data. The low-quality data may bias the model training and evaluation process. Therefore, a high-quality real-world vulnerability

```

1 def test_keypair_list(self):
2 - req = webob.Request.blank('/v1.1/123/os-keypairs')
3 + req = webob.Request.blank('/v1.1/fake/os-keypairs')
4 res = req.get_response(fakes.wsgi_app())
5 self.assertEqual(res.status_int, 200)
6 res_dict = json.loads(res.body)
7 response = {'keypairs': [{'keypair': fake_keypair('FAKE')}]}}
8 self.assertEqual(res_dict, response)
9 ...

```

(a) An example of the tangled patch from CVE-2012-0030.

```

1 void* checked_xmalloc(size_t size){
2     alloc_limit_assert("checked_xmalloc", size);
3     return xmalloc(size);
4 }
5
6 void* xmalloc(size_t size){
7     void *ptr = malloc(size);
8     if (!ptr && (size != 0)){
9         perror("xmalloc: Memory allocation failure");
10        abort();
11    }
12    return ptr;
13 }

```

(b) An example of the inter-procedural vulnerability from CVE-2017-6308.

```

1 Commit message: "fix out-of-bounds read in ttm_put_pages() v2"
2 Commit ID: a66477b0efe511d98dde3e4aeb189790e6f0a39
3 Parent ID: d47703d4ecaa9189d70fb5d151a6883cc44afd3
4 File Path: drivers/gpu/drm/ttm/ttm_page_alloc.c
5
6 - if (!(flags & TTM_PAGE_FLAG_DMA32)) {
7 + if (!(flags & TTM_PAGE_FLAG_DMA32) && (npages - i) >= HPAGE_PMD_NR) {
8     for (j = 0; j < HPAGE_PMD_NR; ++j)
9         if (p++ != pages[i + j])
10             break;
11     ...
12 }

```

(c) An example of the outdated patch from CVE-2019-19927.

Figure 1: Examples for illustrating the challenges of existing datasets. Lines highlighted in green denote added content, red indicates deleted content, yellow represents commit information, and blue identifies the caller and callee.

dataset is important yet under-explored for the vulnerability detection task. In the paper, we focus on building the high-quality dataset by mitigating the following limitations of the existing datasets:

(1) Tangled Patches: Vulnerability patches may contain vulnerability-fixing unrelated code changes, resulting in *tangled patches*. Existing datasets [11–13] generally consider all the code changes in one patch submission related to the vulnerabilities, introducing natural data noise. For the example shown in Figure 1(a), the patch from CVE-2012-0030 includes the code change about path modification, in which the request path parameter of the function `webob.Request.blank()` has been changed (Lines 2-3). Such vulnerability-unrelated changes may be concerned with code refactoring or new feature implementation, and are hard to be distinguished. Therefore, identifying vulnerability-fixing related files from multiple files in one patch presents a challenge.

(2) Lacking Inter-procedural Vulnerabilities: Vulnerabilities in real-world scenarios usually involve calls between multiple files and functions, whereas individual functions alone are not necessarily vulnerable. Existing datasets [11, 13] mainly focus on function-level granularity, ignoring the call information. Figure 1(b) illustrates an example of inter-procedural vulnerability (CVE-2017-6308). The return value of the function `checked_xmalloc()` is passed by the function `xmalloc()` (Line 3) in this example. Given that the parameter of `size` could overflow, there is a potential risk of triggering CWE-190 (Integer Overflow or Wraparound). However, the functions `checked_xmalloc()` and `xmalloc()` are flawless themselves, while existing labeling methods may mark them as vulnerable. Models trained on the vulnerable data without considering inter-procedural call information would be biased, limiting their performance in practical scenarios.

(3) Outdated Patches: Patches may introduce new vulnerabilities and become outdated, while current datasets do not consider the timeliness of the patches. Figure 1(c) shows an original patch and its child patch both from CVE-2019-19927. In the original patch, developers add stronger constraints to the existing conditional statement to avoid CWE-125 (Out-of-bounds Read) [14] (Lines 6-7). However, in the next immediate patch, the loop statement after the conditional statement is changed (Lines 7-8), with the commit message stating “fix start page for huge page check”. Clearly, this additional fix is due to the incompleteness of the original patch, and thus the original patch is outdated and should be filtered out.

In this paper, we propose an automated data collection framework and construct a repository-level high-quality vulnerability dataset named **ReposVul** to address the aforementioned limitations. Our framework consists of three modules: ① **A vulnerability untangling module:** We propose to integrate the decisions of Large Language Models (LLMs) and static analysis tools to distinguish the vulnerability-fixing related files within the patches, given their strong contextual understanding capability and domain knowledge, respectively. ② **A multi-granularity dependency extraction module:** We extract the inter-procedural call relationships of vulnerabilities among the whole repository, aiming to construct multi-granularity information for each vulnerability patch, including file-level, function-level, and line-level information. ③ **A trace-based filtering module:** We first track the submission history of patches based on file paths and commit time. Through analyzing historical information on the patches, we then identify outdated patches by tracing their commit diffs.

In summary, our contributions can be outlined as follows:

- (1) We introduce an automated data collection framework for obtaining vulnerability data. Our framework consists of a vulnerability untangling module to identify vulnerability-fixing related files within tangled patches, a multi-granularity dependency extraction module to construct inter-procedural vulnerabilities, and a trace-based filtering module to recognize outdated patches.
- (2) ReposVul is the first repository-level vulnerability dataset, including large-scale CVE entries representing 236 CWE types across 1,491 projects and four programming languages with detailed multi-granularity patch information.
- (3) Through manual checking and data analysis, ReposVul is high in quality and alleviates the limitations of the existing vulnerability datasets. We have publicly released the source code as well as ReposVul at: <https://github.com/Eshe0922/ReposVul>.

The remaining sections of this paper are organized as follows. Section 2 introduces the framework to collect ReposVul. Section 3 presents the evaluation and experimental results. Section 4 discusses the data application and limitations of ReposVul. Section 5 introduces the background of the OSS vulnerability datasets and detection methods. Section 6 concludes the paper.

2 FRAMEWORK

Figure 2 presents an overview of our data collection framework, which contains four procedures to construct the vulnerability dataset. The framework begins with the *raw data crawling* step, designed to gather vulnerability entries and associated patches, resulting in an initial dataset. This dataset is then processed through the following three key modules to yield the final dataset ReposVul: (1) The **vulnerability untangling module** aims at automatically identifying vulnerability-fixing related files within tangled patches, by jointly considering the decisions from LLMs and static analysis tools. (2) The **multi-granularity dependency extraction module** extracts inter-procedural call relationships associated with vulnerabilities throughout the repository. (3) The **trace-based filtering module** tracks the submission history of patches based on file paths and commit time, aiming to analyze commit diffs corresponding to patches.

2.1 Raw Data Crawling

This phase aims to collect the extensive raw vulnerability data, with detailed entry information illustrated in Table 1. The creation of the initial dataset involves three steps: 1) crawling vulnerability entries from open-source databases, 2) fetching patches associated with the vulnerability entry from multiple platforms, and 3) obtaining detailed information on changed files involved in the patch.

2.1.1 Vulnerability Entry Collection. We collect open-source vulnerabilities from Mend [15] which involves both popular and under-the-radar community resources with extensive vulnerability entries. During the collection, we retrieve CVE entries in chronological order for the identification of outdated patches. We then store these entries in a structured format, as illustrated in the “Vulnerability Entry Information” part of Table 1. Each entry encompasses essential features such as the CVE-ID, CVE description, associated CWE-ID, and other relevant information.

Table 1: Basic information for each entry in ReposVul, including vulnerability entry information, patch information, and related file information.

Features	Description
Vulnerability Entry Information	
CVE-ID	The CVE that the entry belongs to
CWE-ID	The CWE that the entry belongs to
Language	The programming language of the CVE
Resource	Involved links of the CVE
CVE Description	The description of the CVE
Publish-Date	The publish date of the CVE
CVSS	The CVSS score of the CVE
CVE-AV	The attack vector of the CVE
CVE-AC	The attack complexity of the CVE
CVE-PR	The privileges required of the CVE
CVE-UI	The user interaction of the CVE
CVE-S	The scope of the CVE
CVE-C	The confidentiality of the CVE
CVE-I	The integrity of the CVE
CVE-A	The availability of the CVE
CWE Description	The description of the CWE
CWE Solution	Potential solutions of the CWE
CWE Consequence	Common consequences of the CWE
CWE Method	Detection methods of the CWE
Patch Information	
Commit-ID	The commit id of the patch
Commit-Message	The commit message of the patch
Commit-Date	The commit date of the patch
Project	The project that the patch belongs to
Parent Patch	The parent patch of the patch
Child Patch	The child patch of the patch
URL	The API-URL of the patch
Html-URL	The Html-URL of the patch
Related File Information	
File-Name	The name of the file
File-Language	The programming language of the file
Code-Before	The content of the file before fixes
Code-After	The content of the file after fixes
Code-Change	The code changes of the file
Html-URL	The Html-URL of the file

2.1.2 Patch Collection. To comprehensively analyze each vulnerability entry, we collect the corresponding patches. For the majority of the projects, we collect these patches from GitHub [16] and record their Commit-ID and Commit-Message. Additionally, for two special projects, including Android and Chrome, we conduct patch collection on Google Git [17] and bugs.chromium [18], respectively, as some of their patches are not released on GitHub. Detailed patch-level information is summarized in Table 1.

2.1.3 Related File Collection. To extract vulnerability code snippets at file-level and repository-level, we download the entire repository

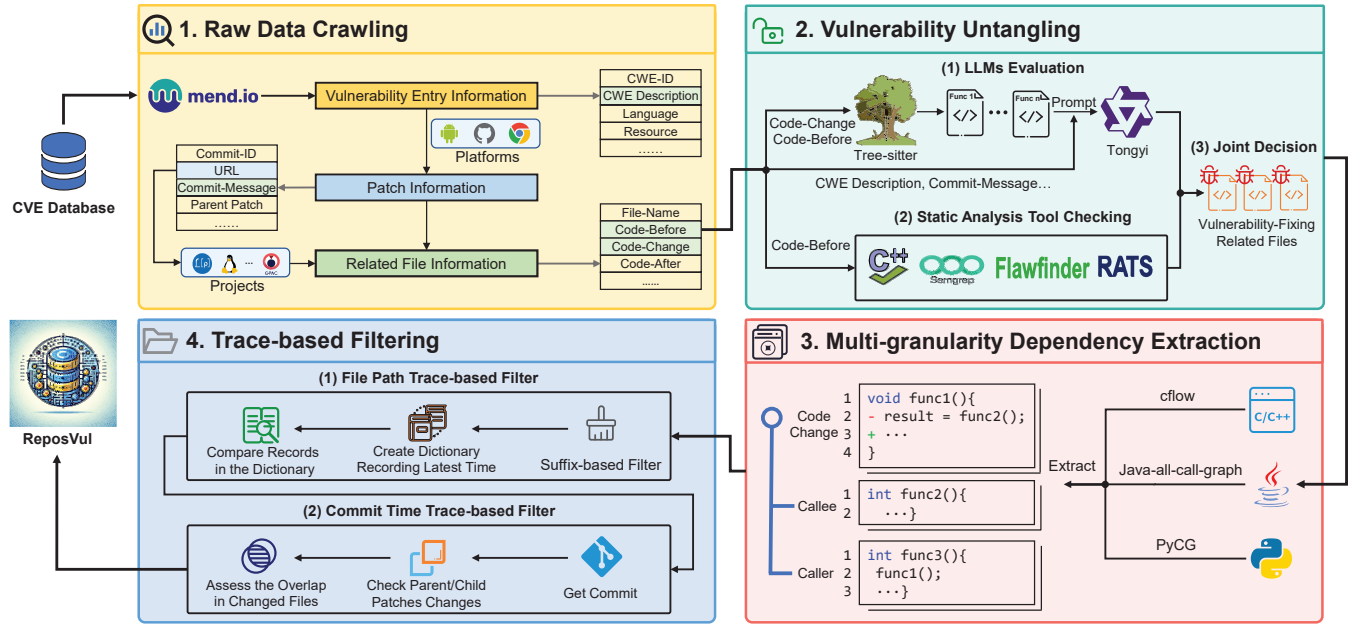


Figure 2: The architecture of our automatic data collection framework.

of the parent patch and child patch associated with each patch using its unique Commit-ID. For each file in the patch, we retrieve its content before and after code changes. In Table 1, we present the details of each related file, including key features like File-Name, Code-Before, and Code-After.

2.2 Vulnerability Untangling Module

The vulnerability untangling module aims to remove vulnerability-fixing unrelated code changes from patches. We employ LLMs for evaluating the relevance between code changes and vulnerability fixes, and static analysis tools for checking vulnerabilities in the code changes, separately. Jointly considering their outputs, we determine whether a changed file is vulnerability-fixing related.

2.2.1 LLMs Evaluation. LLMs possess strong contextual understanding capability and extensive applicability across programming languages. The adaptability of LLMs to different programming languages also ensures that LLMs-based evaluation can be conducted across various code syntaxes and structures. We opt for Tongyi [19] to evaluate the relevance between code changes and vulnerability fixes effectively, considering its accessible free API and support for long context inputs.

To effectively leverage LLMs’ contextual understanding capability, we craft a task-specific prompt to evaluate the relevance of code changes to the corresponding vulnerability fixes, as depicted in Figure 3. This prompt consists of several components: **(1) System prompt:** The LLM acts as an expert, analyzing code vulnerabilities and their corresponding fixes. **(2) Contextual prompt:** The prompt involves four types of contextual information. **① CWE description:** Based on the CWE-ID associated with each patch, we provide a brief vulnerability description. **② CWE solution:** We offer the recommended solutions according to the CWE-ID, which helps LLMs

assess the alignment of code changes with the vulnerability solutions. **③ Commit message:** The commit messages from patches aid LLMs in comprehending the purpose of the fixes. **④ Function:** We utilize the Tree-sitter tool [20] to extract functions affected by the code changes in the file. This prompt contextualizes the code changes, aiding in a precise analysis. **(3) Input Code Change:** The specific code changes made in the file. **(4) Answer prompt:** The LLMs evaluate the relevance of code changes in the file to vulnerability fixes and output “YES” or “NO” indicating whether the code changes are vulnerability-fixing related. Given the prompt with detailed vulnerability and patch information, LLMs can effectively determine the relevance of code changes to vulnerability fixes based on the code context.

2.2.2 Static Analysis Tool Checking. Besides LLMs, we also employ static analysis tools for vulnerability checking. Static analysis tools detect vulnerabilities in code by extracting source code models [21] and employing diverse vulnerability rules, ensuring high code coverage and low false-negative rates. We employ various static analysis tools to check for vulnerabilities in code changes, which integrate the vulnerability rules and expert knowledge from different tools.

We choose four well-established tools, including Cppcheck [22], Flawfinder [23], RATS [24], and Semgrep [25], considering their open-source availability and flexible configuration options, for achieving effective vulnerability detection across multiple programming languages. For different languages, we employ different static analysis tools based on the applicability of the tools. Specifically, for C and C++ files, we use all the four tools. For Python files, our analysis is conducted using RATS and Semgrep; while for Java files, we only use Semgrep. For each file in a patch, we first employ static analysis tools to check whether the before-fixing file contains

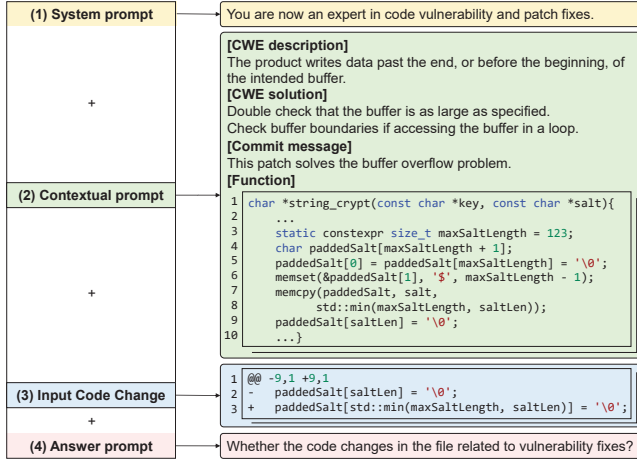


Figure 3: A sample prompt for LLMs to evaluate the relevance of code changes in one file to the vulnerability fixes.

vulnerabilities. We then determine one file as vulnerability-fixing related only if the vulnerabilities detected in the before-fixing version¹ correspond to code changes in the patch.

2.2.3 Joint Decision. We combine the comprehension capabilities of LLMs and the domain knowledge of static analysis tools to identify vulnerability-fixing related files in one patch. The files are labeled as related or not only if both LLMs and static analysis tools reach the same decisions. Files with conflicting results from the LLMs and static analysis tools are excluded for the subsequent processing.

2.3 Multi-granularity Dependency Extraction Module

Existing vulnerability datasets mainly focus on function-level vulnerabilities, ignoring the rich context information. To mitigate this challenge, we extract the inter-procedural call relationships of vulnerabilities throughout the repository and construct multiple-granularity information for each vulnerability patch, including information at repository level, file level, function level, and line level.

2.3.1 Pilot Experience. Figure 1(a) illustrates an example of an inter-procedural CWE-190 (Integer Overflow or Wraparound) vulnerability [26]. Figure 4(a) and Figure 4(b) show two real-world solutions implemented across subsequent commits, which involve changing the caller function and adjusting the callee function, respectively. Specifically, Figure 4(a) demonstrates the first solution, where the function `check_mul_overflow()` is used to verify if the variable `size` is overly large before the `xalloc()` function is called (Lines 3-4). Figure 4(b) illustrates the second solution, which involves a check within the `xalloc()` function itself to assess the size of `size` (Lines 3-4). Therefore, for the code snippets, whether function as callers or callees, inter-procedural vulnerabilities can be introduced. Note that code changes of Figure 4(a) do not cover

¹We aggregate the vulnerabilities detected by the multiple static analysis tools during the processing.

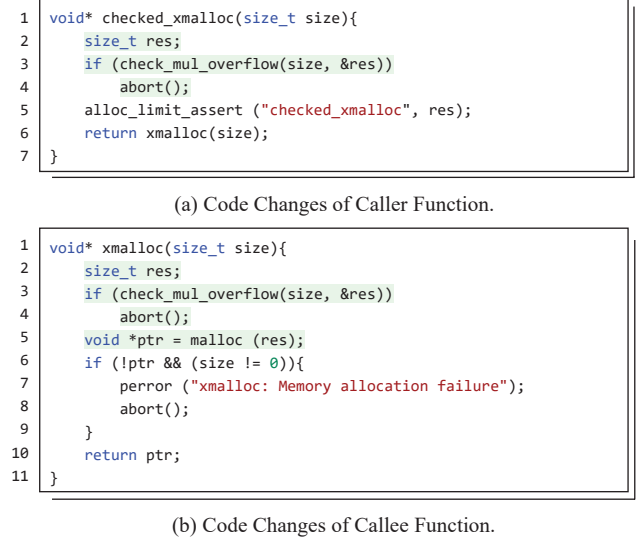


Figure 4: Two solutions to fix the inter-procedural vulnerability. The tokens highlighted in green indicate code changes related to vulnerability fixing.

Algorithm 1: Repository-level Dependency Extraction

```
Input : selLang; // selected programming language
       VulFile; // all vulnerability-fixing related files in a project
Output: callerTree; // caller trees in the project
       calleeTree; // callee trees in the project
1 // initialize the callerTree and calleeTree;
2 callerRoot ← ∅; calleeRoot ← ∅;
3 foreach file ∈ VulFile do
4   project ← getParentProject(file);
5   allFile ← getAllFile(project, selLang);
6   VulSnip ← getSnip(file);
7   foreach snip ∈ VulSnip do
8     callerRoot ← getNearFunc(snip, file);
9     calleeRoot ← getOutFunc(snip, file);
10    // adopt static analysis tools to extract caller and callee chains;
11    callerTree.add(StaticToolExtractor(callerRoot, allFile, 0));
12    calleeTree.add(StaticToolExtractor(calleeRoot, allFile, 1));
13  end
14 end
```

the sixth line, which contains the inter-procedural vulnerability API, `xmalloc()`. Therefore, when capturing the inter-procedural vulnerability from a code snippet, we need to expand the search scope to its top-level functions.

2.3.2 Repository-level Dependency Extraction Algorithm. We develop a dependency extraction algorithm comprising two components: *Repository-level Dependency Extraction* (Algorithm 1) and *Root of Caller and Callee Tree Extraction* (Algorithm 2). Algorithm 1 serves as the primary process for dependency extraction, while Algorithm 2 specifically addresses the extraction of the roots of function caller and callee trees.

Algorithm 1 presents our repository-level dependency extraction framework. It takes the selected programming language and all vulnerability-fixing related files in a project as input and produces

Algorithm 2: Root Extraction for Caller and Callee Trees

```

1 // extract the root of the function callee tree;
2 Function getOutFunc(snip, file):
3   calleeFunc  $\leftarrow \emptyset$ ;
4   // extract all the first layer function of file;
5   outFunc  $\leftarrow$  getTopLevelFunc(file);
6   foreach func  $\in$  outFunc do
7     if snip  $\cap$  func  $\neq \emptyset$  then
8       | calleeFunc.add(snip);
9     end
10  end
11 return calleeFunc
12 // extract the root of the function caller tree;
13 Function getNearFunc(snip, file):
14   // expand the search scope to the top-level functions;
15   callerSnip  $\leftarrow$  getOutFunc(snip, file);
16 return getAPI(callerSnip, file)

```

two outputs: the function caller tree (*callerTree*) and the function callee tree (*calleeTree*). The algorithm works as follows: for each changed code snippet in a vulnerability-fixing related file, we initially identify the root functions for both caller and callee trees, named *callerRoot* and *calleeRoot*, respectively (Lines 8-9). Subsequently, a specific static analysis tool is used to build the caller trees from *callerRoot* and callee trees from *calleeRoot* (Lines 11-12). For different programming languages, we use different tools: cflow [27] for C/C++, Java-all-call-graph [28] for Java, and PyCG [29] for Python, considering these tools are specifically designed for certain programming languages.

Algorithm 2 is designed to identify the roots of function caller and callee trees. For the root of the function callee tree extraction, we identify all the top-level functions within the file (Line 5). Next, we retrieve the top-level functions that overlap with the provided code snippet (Lines 6-10). For the root of the function caller tree extraction, we first expand the scope of API extraction to the top-level functions containing the given code snippet (Line 15). We then leverage the Tree-sitter tool [20] to extract all relevant APIs in the scope (Line 16).

2.3.3 Multi-granularity Code Snippet. To identify inter-procedural and intra-procedural vulnerabilities and facilitate the localization of vulnerabilities, we construct multiple-granularity information for each patch, including repository-level, file-level, function-level, and line-level. As shown in Table 2, each patch contains multi-granularity information.

Repository-level: For each vulnerability-fixing related file, we employ Algorithm 1 to extract the inter-procedural call relationships of vulnerabilities among the whole repository.

File-level: We consider the vulnerability-fixing related files before and after code changes as vulnerable and non-vulnerable, respectively. For vulnerability-fixing unrelated files, we consider both the files before and after code changes as non-vulnerable.

Function-level: For each function affected by code changes, if the function is defined in vulnerability-fixing related files, we consider the function before and after code changes as vulnerable and non-vulnerable, respectively; if the function is defined in vulnerability-fixing unrelated files, we consider both the function before and after code changes as non-vulnerable. For each

Table 2: Multi-granularity information for each patch in ReposVul, including file-level, function-level, file-level, and repository-level information.

Features	Description
Line-level	
Line	The content of the code line
Line-Number	The line number of the code line
Function-level	
Function	The content of the function
Target	The vulnerability label of the function
File-level	
LLMs-Evaluate	LLMs evaluation results
Static-Check	Static analysis tools checking results
Target	The vulnerability label of the file
Repository-level	
Inter-procedural Code	The content of the inter-procedural code
Target	The vulnerability label of the code snippet

function unaffected by code changes, we consider the function as non-vulnerable.

Line-level: We extract the line changes and their line numbers from code changes in the patch and leverage these line changes as a precise detection target.

2.4 Trace-based Filtering Module

Patches may introduce vulnerabilities and become outdated, but the existing datasets do not distinguish outdated patches, posing a potential risk to data quality. In this module, we initially track the submission history of patches based on file paths and commit time. Through analyzing patches' historical information, we then recognize outdated patches by tracing their commit diffs.

2.4.1 File Path Trace-based Filter. We initially filter noise files according to the suffixes. For example, some files such as description documentation (suffixed with *.md*, *.rst*), data (suffixed with *.json*, *.svg*), change logs (suffixed with *.ChangeLog*), and output files (suffixed with *.out*), are generally unrelated to functionality implementation. For the remained files, we create a dictionary associating file paths with their most recent submission dates in our collected vulnerability patches. We then review the submission date of each file, comparing it to the latest date recorded in the dictionary. If the file's submission date is not the most recent, we retain the file for the subsequent commit time trace-based filter. Otherwise, we filter it out. As shown in Figure 1(c), the file *ttm_page_alloc.c* still contains vulnerabilities after the first code changes. We create the entry in the dictionary based on the file's path "*ttm_page_alloc.c*" and its latest submission date. According to the file's submission dates, we consider the earlier submitted file (commit id a66477b) as vulnerable.

2.4.2 Commit Time Trace-based Filter. We first retrieve each original patch's parent patch and child patch based on its commit time. We then assess whether the files changed within the parent patch and child patch overlap with those changed by the original patch. If there is an overlap within the retained files by the file path trace-based filter, we recognize the original patch as outdated. As illustrated in Figure 1(c), the original patch (commit id a66477b) and its child patch (commit id ac1e516) change the same file `ttm_page_alloc.c` due to the incompleteness of the original patch. By comparing the overlapping between the changed files in the original patch and its child patch, we can find the same changed file `ttm_page_alloc.c`. Since the file is retained by the file path trace-based filter, we consider the original patch as outdated.

3 EVALUATION AND EXPERIMENTAL RESULTS

In this section, we illustrate the advantages of ReposVul and focus on the following four Research Questions (RQs):

- RQ1:** What are the advantages of ReposVul compared to the existing vulnerability datasets?
- RQ2:** What is the quality of data labels in ReposVul?
- RQ3:** To what extent do the selection of LLMs and prompt design in the vulnerability untangling module affect the label quality?
- RQ4:** How does ReposVul perform in filtering outdated patches?

3.1 RQ1: Advantages of ReposVul

To answer RQ1, we compare ReposVul with six widely-used vulnerability detection datasets [11–13, 30–32] from several aspects, including the granularity of vulnerabilities, number of CWE types, vulnerability labeling methods, outdated patches recognition, and additional information. As shown in Table 3, ReposVul has the following advantages compared to current datasets:

Multi-granularity information: Compared to other datasets that only contain vulnerabilities at the line-level, function-level, and file-level, ReposVul contains more comprehensive granularities, including repository-level, file-level, function-level, and line-level vulnerabilities, which considers inter-procedural vulnerabilities and provides information that is more than a single patch. As shown in Table 4, ReposVul comprises 14,706 files from 6,897 patches. Specifically, ReposVul encompasses 212,790 functions in C, 20,302 in C++, 2,816 in Java, and 26,308 in Python, respectively.

Extensive CWE coverage: As shown in Table 3, ReposVul covers more CWE types than all other datasets. ReposVul covers 149 CWE types in C, 105 in C++, 129 in Java, and 159 in Python, respectively. ReposVul encompasses CWE types that extend across various programming languages since some CWE types are not language-specific. It indicates that ReposVul provides more comprehensive data than existing benchmarks.

Effective labeling methods: The previous work [10] has identified the noisy data problem in the existing datasets by the current labeling method. In this paper, ReposVul proposes the vulnerability untangling module for improving the vulnerability data quality. It involves the vulnerability rules and domain knowledge from static analysis tools and the strong contextual understanding capability from LLMs.

Recognition of outdated patches: Current vulnerability datasets do not distinguish outdated patches. ReposVul employs the trace-based filtering module to recognize potential outdated patches. The trace-based filtering module integrates file path trace-based filter and commit time trace-based filter to provide labels for outdated patches.

Specific richness of additional information: ReposVul contains the richest additional information, including CVE descriptions, CVSS, and patch submission history illustrated in Table 1, and static analysis information illustrated in Table 2. Comprehensive information on vulnerabilities enables developers and researchers to take effective measures for vulnerability detection.

Answer to RQ1: Compared to the existing datasets, ReposVul incorporates multi-granularity code snippets and the most extensive range of CWE types. It also employs an effective method for labeling vulnerability data and provides annotations for outdated patches, along with other rich additional information.

3.2 RQ2: Label Quality of ReposVul

To answer the RQ2, we first conduct experiments to assess the labeling method of ReposVul (i.e., the vulnerability untangling module in the data collection framework). Then, we compare the label quality of ReposVul with that of the previous datasets.

Comparison with LLMs and static analysis tools: We compare the label quality of ReposVul with that obtained by LLMs and static analysis tools separately. The results are presented in Table 5. Specifically, we randomly select 50 CVE cases for each programming language. We recruit three academic researchers as participants, and each of them possesses over five years of software vulnerability detection experience. According to the vulnerability description and the commit message of the patch, the participants manually label the code changes of the file in the patch as “Yes” or “No”, corresponding to whether the code changes of the file are relevant to the vulnerability fix or not, respectively. After assessing, participants reach agreements on 96% for the cases. For the remaining discrepancies, participants negotiate and reach a consensus. As shown in Table 5, we observe that the proposed labeling method achieves the highest accuracy of 85% across the four programming languages on average, exceeding LLMs by 10% and static analysis tools by 5.5%.

Comparison with the existing datasets: The previous work [10] has shown that 20–71% of vulnerability labels are inaccurate in the state-of-the-art OSS vulnerability datasets. Analyzing the results in Table 5, we observe that ReposVul shows the labeling accuracy at 85%, 90%, 85%, and 80% on C, C++, Java, and Python, respectively. Compared with the existing OSS vulnerability datasets as reported in [10], ReposVul achieves relatively higher accuracy across the four programming languages, especially for the C++ programming language with an outstanding accuracy at 90%. Our observations indicate that ReposVul's label quality is better than previous datasets due to the integration of contextual understanding capability from LLMs and domain knowledge from static analysis tools.

Table 3: Comparison between ReposVul and six widely-used vulnerability detection datasets.

Baseline	Granularity				CWE Types	Labeling Method	Outdated Patch Recognition	Additional Information
	Line	Function	File	Repository				
BigVul [12]	✓	✓	✗	✗	91	Commit Code Diff	✗	✓
D2A [30]	✓	✓	✗	✗	-	Static Analysis	✗	✓
Devign [11]	✗	✓	✗	✗	-	Manually	✗	✗
Reveal [13]	✗	✓	✗	✗	-	Commit Code Diff	✗	✗
CrossVul [31]	✗	✗	✓	✗	168	Commit Code Diff	✗	✓
DiverseVul [32]	✗	✓	✗	✗	150	Commit Code Diff	✗	✓
ReposVul	✓	✓	✓	✓	236	LLMs + Static Analysis	✓	✓

Table 4: Statistics of ReposVul.

Languages	CVE Entries	Patches	Files	Functions
C	3,565	4,010	7,515	212,790
C++	631	689	1,506	20,302
Java	786	888	2,925	2,816
Python	1,152	1,310	2,760	26,308
Total	6,134	6,897	14,706	262,216

Table 5: Results of manual examination in 50 cases per programming language.

Language	Method	Accuracy
C	LLMs	70%
	Static analysis tool	82%
	ReposVul	85%
C++	LLMs	80%
	Static analysis tool	84%
	ReposVul	90%
Java	LLMs	78%
	Static analysis tool	78%
	ReposVul	85%
Python	LLMs	72%
	Static analysis tool	74%
	ReposVul	80%

Answer to RQ2: After manual checking of a subset of labelled data, the label quality of ReposVul outperforms the existing datasets, achieving accuracy of 85%, 90%, 85%, and 80% on C, C++, Java, and Python, respectively. The proposed labeling method also performs better than LLMs and static analysis tools applied separately.

3.3 RQ3: Influence of Different LLMs and Prompt Design on Label Quality

To answer the RQ3, we conduct the experiments to analyze the impact of LLM and prompt in the vulnerability untangling module on the label quality of ReposVul. In this section, we randomly select 20 CVE cases, encompassing code changes that may or may not be associated with vulnerability fixes. We recruit the same three academic researchers who participate in the manual annotation of Section 3.2 for labeling. Participants manually label the code changes as “Relevant” and “Irrelevant”, respectively. After assessing, participants reach agreements on 100% of the 20 cases.

LLM selection: We present the nine investigated LLMs in Table 6. ChatGLM [33] and Baichuan2 [34] are open-source LLMs from THUDM and baichuan-ai, respectively, trained on Chinese and English corpora. Llama2 [35] boasts a larger training dataset compared to Llama [36]. CodeLlama [37] is trained and fine-tuned using code-related data based on Llama2. ChatGPT [38] and GPT-4 [39] are commercial LLMs from OpenAI. Tongyi [19] is developed by Alibaba Cloud, demonstrating excellent performance across multiple tasks. The experimental results are presented in Table 6. The results show that GPT-4 and Tongyi achieve the highest accuracy of 85%, identifying all code changes related to vulnerability fixes and the majority of code changes unrelated to vulnerability fixes. Llama2, on the other hand, considers all code changes as relevant to vulnerability fixes and thus cannot effectively distinguish tangled patches. The other LLMs’ accuracy does not exceed 80%. Considering that Tongyi [19] has the highest accuracy with free API accessibility, we choose this LLM for labeling.

Prompt design: The previous work [6] has demonstrated that the code-related task performance is largely influenced by the prompt. We also construct four variations, including prompt without CWE description (*i.e.* w/o CWE Description), CWE solution (*i.e.* w/o CWE Solution), commit message (*i.e.* w/o Commit Message), and function of the code changes (*i.e.* w/o Function). As illustrated in Table 7, the prompt used by ReposVul achieves the highest accuracy of 85%. The four variations result in accuracy decreases of 30%, 15%, 5%, and 5%, respectively. The results indicate that the CWE description has the greatest impact on LLMs owing to its rich vulnerability information compared to the CWE solution and the function of the code changes. The relatively small impact of

Table 6: Performance of different LLMs evaluation in 20 CWE cases. The numbers split by “/” in the “Relevant” (or “Irrelevant”) column represent the number of correct and incorrect responses in relevant (or irrelevant) code changes. The symbol “-” indicates that the corresponding statistic is unknown.

Model	Size	Relevant	Irrelevant	Accuracy
Llama2-7b [35]	7B	4/0	0/16	20%
Llama2-13b [35]	13B	4/0	0/16	20%
Baichuan2 [34]	7B	1/3	9/7	50%
CodeLlama-7b [37]	7B	3/1	9/7	60%
CodeLlama-13b [37]	13B	3/1	10/6	65%
ChatGLM [33]	6B	2/2	11/5	65%
ChatGPT [38]	20B	3/1	12/4	75%
GPT-4 [39]	-	4/0	13/3	85%
Tongyi [19]	-	4/0	13/3	85%

Table 7: Performance of different prompts evaluation in 20 CWE cases. The numbers split by “/” in the “Relevant” (or “Irrelevant”) column represent the number of correct and incorrect responses in relevant (or irrelevant) code changes.

Prompt	Relevant	Irrelevant	Accuracy
w/o CWE Description	3/1	8/8	55%
w/o CWE Solution	3/1	11/5	70%
w/o Commit Message	3/1	13/3	80%
w/o Function	4/0	12/4	80%
ReposVul	4/0	13/3	85%

the commit message on the LLMs may be due to inaccurate and redundant information in the commit messages.

Answer to RQ3: Among different LLMs, GPT-4 and Tongyi achieve the best accuracy of 85% in labeling the relevance of code changes to vulnerability fixes. In prompt design, CWE description and CWE solution have a relatively considerable impact on the LLMs’ decision. The proposed prompt achieves the highest accuracy of 85%.

3.4 RQ4: Performance of Filtering Outdated Patches in ReposVul

To answer the RQ4, we present the statistics of recognized outdated patches by ReposVul from different aspects, including CWEs, time, projects, and programming languages.

CWEs: Figure 5(a) shows the distribution of outdated patches and total patches of the Top-10 CWEs by the number of outdated patches. We observe that CWE-119 (Improper Restriction of Operations within the Bounds of a Memory Buffer) [40], CWE-787 (Out-of-bounds Write) [41], and CWE-125 (Out-of-bounds Read) [14] contain the most outdated patches, including 121, 108, and 97 outdated patches, respectively. These CWEs are associated with buffer

operations, indicating that developers are more prone to introducing new vulnerabilities when modifying code snippets related to buffers, thereby resulting in outdated patches.

Time: Figure 5(b) depicts the distribution of outdated patches and total patches throughout the past decade. It reveals a consistent upward trend in the overall number of patches associated with OSS, ascending from 264 in 2014 to 1,132 in 2022. Concurrently, the count of outdated patches has seen a modest increase, progressing from 53 in 2014 to 118 in 2022. Notably, in 2023, both metrics exhibit a decline, which can be attributed to undisclosed vulnerabilities. Moreover, the ratio of outdated patches to total patches demonstrates a notable decrease, plummeting from 20.07% in 2014 to 10.42% in 2022. This decline indicates a heightened emphasis by developers on enhancing the robustness of patches.

Projects: Figure 5(c) presents the Top-10 projects ranked by the number of outdated patches. The proportion of outdated patches is highest in the Linux project, attaining a rate of 16.05%. Following closely are ImageMagick and Vim, exhibiting proportions of 10.26% and 6.58%, respectively. The heightened prevalence in Linux can be attributed to its expansive project scope and the vast number of files, making it susceptible to the introduction of new vulnerabilities during the submission of patches. Notably, ImageMagick and Vim manifest a considerable number of outdated patches despite their smaller project sizes. It may be due to delays of patch information maintained by the security vendors.

Programming languages: Figure 5(d) illustrates the distribution of outdated patches categorized by programming languages. Among these languages, C constitutes the predominant share at 70.7%, followed by Java at 14.6%, Python at 8.1%, and C++ at 6.6%. The heightened occurrence within the C programming language is associated with improper buffer operations related to CWE-119, CWE-787, and CWE-125. The vulnerabilities are notably attributed to the incorporation of flexible arrays and the absence of built-in boundary-checking mechanisms.

Answer to RQ4: Within the identified outdated patches, those associated with buffer operations comprise the majority. The quantity of outdated patches increases over time, yet their proportion in the total patches diminishes. Linux and C have the highest proportions of outdated patches by 16.05% and 70.7%, respectively, across projects and programming languages.

4 DISCUSSION

4.1 Data Application

ReposVul is the first repository-level vulnerability dataset across multiple programming languages. ReposVul can be used for addressing a range of OSS vulnerability-related tasks. We advocate for the utilization of ReposVul as a benchmark, promoting a standardized and practical evaluation of model performance.

Multi-granularity vulnerability detection: ReposVul covers multi-granularity information, including repository-level, file-level, function-level, and line-level features. Researchers can leverage these features to detect inter-procedural and intra-procedural vulnerabilities. In addition, the experiments have demonstrated that

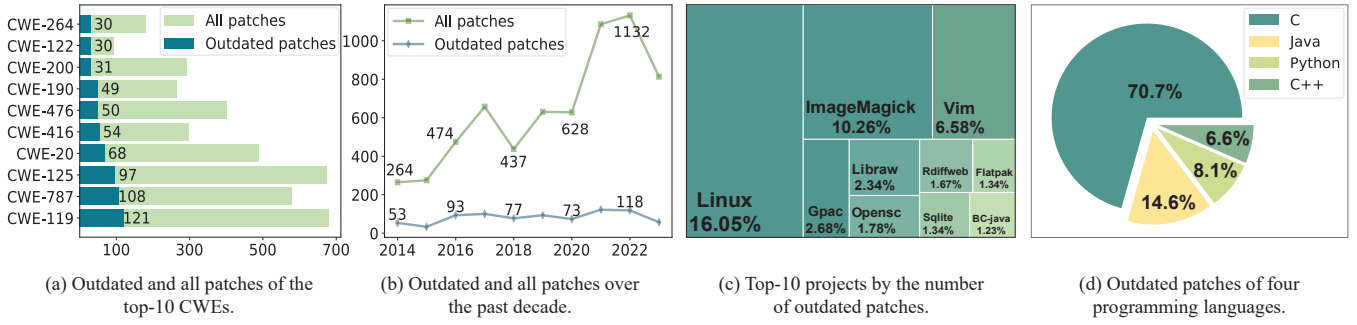


Figure 5: Outdated patches about CWEs, time, projects, and programming languages.

ReposVul outperforms the existing state-of-the-art datasets in label quality. It supports DL-based vulnerability detection methods for better training in the future.

Patch management: ReposVul covers a rich set of patch information including the submission date, parent patches, and historical submission information of vulnerability patches. Researchers and practitioners can utilize this timely information to learn the patching process of existing software vulnerabilities and identify outdated patches.

Vulnerability repair: ReposVul provides CVE descriptions, CWE solutions, CWE consequences, and CWE methods. This information aids developers to better understand the causes and solutions of vulnerabilities. Future research is expected to incorporate the rich contextual information for automatically repairing OSS vulnerabilities.

4.2 Threats and Limitations

One threat to validity comes from the collecting source platforms. During the collecting process, we collect ReposVul from GitHub, Google Git, and bugs.chromium, which leads to missing projects that are hosted on other platforms in ReposVul.

The second threat to validity is the programming language. We only extract repository-level dependency for four widely-used programming languages due to the language-specific features. However, vulnerabilities also exist in other languages, such as JavaScript, Go, and PHP. We plan to extract repository-level dependency for more languages in future work.

Another validity to the threat comes from the collecting time, we only collect the CVEs from 2010 and the previous CVEs are not included in ReposVul, which causes some vulnerabilities may be discovered and fixed in previous years.

5 RELATED WORK

5.1 OSS Vulnerability Dataset

The previous works utilize different methods for constructing OSS vulnerability datasets. It consists of manual checking-based, commit code diff-based, and static analysis tool generation. Manual checking-based datasets [42–45] utilize test cases crafted artificially. For example, SARD [46] includes samples from student-authored and industrial production. Pradel et al. [47] employ code transformation to convert non-vulnerable samples into vulnerable

ones. However, the manual checking-based methods face limitations in labeling efficiency. Commit code diff-based datasets [12, 13, 32] gather patches from open-source repositories and extract vulnerability data from code changes in the patches. CVEfixes [48] fetches vulnerable entries from the National Vulnerability Database (NVD) [49] and corresponding fixes based on entries' reference links. CrossVul [31] generates an authentic dataset spanning 40 programming languages and 1,675 projects, while it only provides file-level source code. In addition, some datasets use static analysis tools [22, 24, 50] to label vulnerability data. For example, Russell et al. [51] integrate the results of multiple static analysis tools to determine whether vulnerabilities exist in code snippets. D2A [30] collects vulnerabilities from six open-source projects and employs the Infer [50] to label data.

However, all these existing vulnerability datasets face challenges in label quality and lack inter-procedural vulnerability. In this paper, we propose the vulnerability untangling module for distinguishing vulnerability-fixing related code changes from tangled patches. We also propose the multi-granularity dependency extraction module for capturing the inter-procedural call relationships.

5.2 OSS Vulnerability Detection

OSS vulnerability detection is essential to identify security flaws and maintain software security. The existing OSS vulnerability detection methods consist of program analysis-based methods [4, 5, 21, 52] and learning-based vulnerability detection methods [7, 8, 53–56].

The program analysis-based methods utilize expert knowledge in extracting features to identify vulnerabilities, including data flow analysis [57] and symbolic execution [58]. Data flow analysis [23, 24, 59–61] tracks the data flow along the execution paths of the program to obtain status information at program points, thus detecting vulnerabilities based on the security of program points. Symbolic execution [62–64] employs symbolic inputs instead of actual inputs and detects program vulnerabilities by determining whether symbolic expressions satisfy constraints.

The learning-based methods can be classified into sequence-based [6, 9, 65, 66] and graph-based methods [67–70] due to the representation of the source code. Sequence-based methods transform code into token sequences. For example, VulDeePecker [71] utilizes code gadgets to represent programs and employs a Bidirectional Long Short-Term Memory (BiLSTM) network for training. μ VulDeePecker [72] combines three BiLSTM networks, enabling

the detection of various types of vulnerabilities. Russell et al. [51] integrate convolutional neural networks and recurrent neural networks for feature extraction, utilizing a random forest classifier for capturing vulnerability patterns. Graph-based methods represent code as graphs and use graph neural networks for software vulnerability detection. Qian et al. [73] employ attributed control flow graphs to construct a vulnerability search engine. Devign [11] adopts gated graph neural networks to process multiple directed graphs generated from source code.

However, all these learning-based methods require a large amount of high-quality labeled samples to achieve good performance. In this paper, we construct a repository-level high-quality dataset named ReposVul to promote the model training.

6 CONCLUSION

In this paper, we propose an automated data collection framework and construct the first repository-level vulnerability dataset named ReposVul. Our framework consists of a vulnerability untangling module for identifying tangled patches, a multi-granularity dependency extraction module for extracting inter-procedural vulnerabilities, and a trace-based filtering module for recognizing outdated patches. ReposVul covers 6,134 CVE entries across 1,491 projects and four programming languages. After comprehensive data analysis and manual checking, ReposVul proves to be high-quality and widely applicable compared with the existing vulnerability dataset.

Our source code, as well as ReposVul, are available at <https://github.com/Eshe0922/ReposVul>.

ACKNOWLEDGMENTS

This research is supported by National Key R&D Program of China (No. 2022YFB3103900), National Natural Science Foundation of China under project (No. 62002084), Natural Science Foundation of Guangdong Province (Project No. 2023A1515011959), Shenzhen Basic Research (General Project No. JCYJ20220531095214031), Shenzhen International Science and Technology Cooperation Project (No. GJHZ20220913143008015), and the Major Key Project of PCL (Grant No. PCL2022A03).

REFERENCES

- [1] Cisco. "Cisco IOS XE Software Web UI Elevation of Privilege Vulnerability", 2023. <https://nvd.nist.gov/vuln/detail/CVE-2023-20198>.
- [2] Roberto Baldoni, Emilio Coppa, Daniele Cono D'Elia, Camil Demetrescu, and Irene Finocchi. A survey of symbolic execution techniques. *ACM Comput. Surv.*, 51(3):50:1–50:39, 2018.
- [3] Richard Bonett, Kaushal Kafle, Kevin Moran, Adwait Nadkarni, and Denys Poshyvanyk. Discovering flaws in security-focused static analysis tools for android using systematic mutation. In *USENIX Security Symposium*, pages 1263–1280. USENIX Association, 2018.
- [4] Shuitao Gan, Chao Zhang, Peng Chen, Bodong Zhao, Xiaojun Qin, Dong Wu, and Zuoning Chen. GREYONE: data flow sensitive fuzzing. In *USENIX Security Symposium*, pages 2577–2594. USENIX Association, 2020.
- [5] Craig Beaman, Michael Redbourne, J. Darren Mummery, and Saqib Hakak. Fuzzing vulnerability discovery techniques: Survey, challenges and future directions. *Comput. Secur.*, 120:102813, 2022.
- [6] Chaozheng Wang, Yuanhang Yang, Cuiyun Gao, Yun Peng, Hongyu Zhang, and Michael R. Lyu. No more fine-tuning? an experimental evaluation of prompt tuning in code intelligence. In Abhik Roychoudhury, Cristian Cadar, and Miryung Kim, editors, *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2022, Singapore, Singapore, November 14–18, 2022*, pages 382–394. ACM, 2022.
- [7] Zhen Li, Deqing Zou, Shouhuai Xu, Hai Jin, Yawei Zhu, and Zhaoxuan Chen. Sysevr: A framework for using deep learning to detect software vulnerabilities. *IEEE Trans. Dependable Secur. Comput.*, 19(4):2244–2258, 2022.
- [8] Song Wang, Taiyue Liu, and Lin Tan. Automatically learning semantic features for defect prediction. In *ICSE*, pages 297–308. ACM, 2016.
- [9] Hoa Khanh Dam, Truyen Tran, Trang Pham, Shien Wee Ng, John Grundy, and Aditya Ghose. Automatic feature learning for predicting vulnerable software components. *IEEE Trans. Software Eng.*, 47(1):67–85, 2021.
- [10] Roland Croft, Muhammad Ali Babar, and M. Mehdi Kholoosi. Data quality for software vulnerability datasets. In *ICSE*, pages 121–133. IEEE, 2023.
- [11] Yaqin Zhou, Shangqing Liu, Jing Kai Siow, Xiaoning Du, and Yang Liu. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. In *NeurIPS*, pages 10197–10207, 2019.
- [12] Jiahao Fan, Yi Li, Shaohua Wang, and Tien N. Nguyen. A C/C++ code vulnerability dataset with code changes and CVE summaries. In *MSR*, pages 508–512. ACM, 2020.
- [13] Saikat Chakraborty, Rahul Krishna, Yangruibo Ding, and Baishakhi Ray. Deep learning based vulnerability detection: Are we there yet? *IEEE Trans. Software Eng.*, 48(9):3280–3296, 2022.
- [14] CWE-125: Out-of-bounds Read, [n.d.]. <https://cwe.mitre.org/data/definitions/125.html>.
- [15] WhiteSource. "Mend bolt", 2023. <https://www.mend.io/free-developer-tools/>.
- [16] "GitHub", 2023. <https://github.com/>.
- [17] "GoogleGit", 2023. <https://code.google.com/source/git/>.
- [18] "Bugs.chromium", 2023. <https://bugs.chromium.org/p/chromium/issues/list>.
- [19] Alibaba Cloud. "Tongyi Qianwen", [n.d.]. <https://qianwen.aliyun.com/>.
- [20] "Tree-sitter", [n.d.]. <https://tree-sitter.github.io/tree-sitter/>.
- [21] Melina Kulenovic and Dzenana Donko. A survey of static code analysis methods for security vulnerabilities detection. In *MIPRO*, pages 1381–1386. IEEE, 2014.
- [22] Cppcheck-team. "Cppcheck", [n.d.]. <http://cppcheck.sourceforge.net/>.
- [23] D. A. Wheeler. "Flawfinder", [n.d.]. <https://dwheller.com/flawfinder/>.
- [24] andrew d. "rough-auditing-tool-for-security", 2023. <https://github.com/andrew-d/rough-auditing-tool-for-security>.
- [25] r2c. "Semgrep", 2021. <https://semgrep.dev>.
- [26] CWE-190: Integer Overflow or Wraparound, [n.d.]. <https://cwe.mitre.org/data/definitions/190.html>.
- [27] Sergey Poznyakoff. "GNU cflow", 2005. <https://www.gnu.org/software/cflow/>.
- [28] "java-all-call-graph", [n.d.]. <https://github.com/Adrnistrator/java-all-call-graph>.
- [29] Vitalis Salis, Thodoris Sotiropoulos, Panos Louridas, Diomidis Spinellis, and Dimitris Mitropoulos. Pycg: Practical call graph generation in python. In *ICSE*, pages 1646–1657. IEEE, 2021.
- [30] Yunhui Zheng, Saurabh Pujar, Burn L. Lewis, Luca Buratti, Edward A. Epstein, Bo Yang, Jim Laredo, Alessandro Morari, and Zhong Su. D2A: A dataset built for ai-based vulnerability detection methods using differential analysis. In *ICSE (SEIP)*, pages 111–120. IEEE, 2021.
- [31] Georgios Nikitopoulos, Konstantina Dritsa, Panos Louridas, and Dimitris Mitropoulos. Crossvul: a cross-language vulnerability dataset with commit data. In *ESEC/SIGSOFT FSE*, pages 1565–1569. ACM, 2021.
- [32] Yizheng Chen, Zhoujie Ding, Lanya Alowain, Xinyun Chen, and David A. Wagner. Diversevul: A new vulnerable source code dataset for deep learning based vulnerability detection. In *RAID*, pages 654–668. ACM, 2023.
- [33] Aohan Zeng, Xiao Liu, Zhengxiao Du, Zihan Wang, Hanyu Lai, Ming Ding, Zhuoyi Yang, Yifan Xu, Wendi Zheng, Xiao Xia, Weng Lam Tam, Zixuan Ma, Yufei Xue, Jidong Zhai, Wenguang Chen, Peng Zhang, Yuxiao Dong, and Jie Tang. GLM-130B: an open bilingual pre-trained model. *CoRR*, abs/2210.02414, 2022.
- [34] Aiyuan Yang, Bin Xiao, Bingning Wang, Borong Zhang, Ce Bian, Chao Yin, Chenxu Lv, Da Pan, Dian Wang, Dong Yan, Fan Yang, Fei Deng, Feng Wang, Feng Liu, Guangwei Ai, Guosheng Dong, Haizhou Zhao, Hang Xu, Haoze Sun, Hongda Zhang, Hui Liu, Jiaming Ji, Jian Xie, JunTao Dai, Kun Fang, Lei Su, Liang Song, Lifeng Liu, Liyun Ru, Luyao Ma, Mang Wang, Mickel Liu, MingAn Lin, Nuolan Nie, Peidong Guo, Ruiyang Sun, Tao Zhang, Tianpeng Li, Tianyu Li, Wei Cheng, Weipeng Chen, Xiangrong Zeng, Xiaochuan Wang, Xiaoxi Chen, Xin Men, Xin Yu, Xuehai Pan, Yanjun Shen, Yiding Wang, Yiyi Li, Youxin Jiang, Yuchen Gao, Yupeng Zhang, Zenan Zhou, and Zhiying Wu. Baichuan 2: Open large-scale language models, 2023.
- [35] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajwal Bhargava, Shriti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models, 2023.

- [36] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models, 2023.
- [37] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaojing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. Code llama: Open foundation models for code, 2023.
- [38] OpenAI. "ChatGPT", [n.d.]. <https://openai.com/blog/chatgpt>.
- [39] OpenAI. "GPT4", [n.d.]. <https://openai.com/research/gpt-4>.
- [40] CWE-119: Improper Restriction of Operations within the Bounds of a Memory Buffer, [n.d.]. <https://cwe.mitre.org/data/definitions/119.html>.
- [41] CWE-787: Out-of-bounds Write, [n.d.]. <https://cwe.mitre.org/data/definitions/787.html>.
- [42] Aidong Xu, Tao Dai, Huajun Chen, Zhe Ming, and Weining Li. Vulnerability detection for source code using contextual LSTM. In *ICSAI*, pages 1225–1230. IEEE, 2018.
- [43] Xu Duan, Jingzheng Wu, Shouling Ji, Zhiqing Rui, Tianyue Luo, Mutian Yang, and Yanjun Wu. Vulsniper: Focus your attention to shoot fine-grained vulnerabilities. In *IJCAI*, pages 4665–4671. ijcai.org, 2019.
- [44] Nicholas Saccante, Josh Dehlinger, Lin Deng, Suranjan Chakraborty, and Yin Xiong. Project achilles: A prototype tool for static method-level vulnerability detection of java source code using a recurrent neural network. In *ASE Workshops*, pages 114–121. IEEE, 2019.
- [45] Hantao Feng, Xiaotong Fu, Hongyu Sun, He Wang, and Yuqing Zhang. Efficient vulnerability detection based on abstract syntax tree and deep learning. In *INFOCOM Workshops*, pages 722–727. IEEE, 2020.
- [46] NIST. SARD: Software assurance reference dataset., 2022. <https://samate.nist.gov/SRD/index.php>.
- [47] Michael Pradel and Koushik Sen. Deepbugs: a learning approach to name-based bug detection. *Proc. ACM Program. Lang.*, 2(OOPSLA):147:1–147:25, 2018.
- [48] Guru Prasad Bhandari, Amara Naseer, and Leon Moonen. Cvefixes: automated collection of vulnerabilities and their fixes from open-source software. In *PROMISE*, pages 30–39. ACM, 2021.
- [49] NIST. "National Vulnerability Database (NVD)", 2022. <https://nvd.nist.gov/>.
- [50] Facebook. "Infer static analyzer", [n.d.]. <https://fbinfer.com/>.
- [51] Rebecca L. Russell, Louis Y. Kim, Lei H. Hamilton, Tomo Lazovich, Jacob Harer, Onur Ozdemir, Paul M. Ellingwood, and Marc W. McConley. Automated vulnerability detection in source code using deep representation learning. In *ICMLA*, pages 757–762. IEEE, 2018.
- [52] Jannik Powny, Felix Schuster, Lukas Bernhard, Thorsten Holz, and Christian Rossow. Leveraging semantic signatures for bug search in binary programs. In *ACSAC*, pages 406–415. ACM, 2014.
- [53] Xin-Cheng Wen, Xinchen Wang, Cuiyun Gao, Shaohua Wang, Yang Liu, and Zhaoquan Gu. When less is enough: Positive and unlabeled learning model for vulnerability detection. In *ASE*, pages 345–357. IEEE, 2023.
- [54] Xiaomeng Wang, Tao Zhang, Runpu Wu, Wei Xin, and Changyu Hou. CPGVA: code property graph based vulnerability analysis by deep learning. In *ICAIT*, pages 184–188. IEEE, 2018.
- [55] Xin-Cheng Wen, Cuiyun Gao, Jiaxin Ye, Zhihong Tian, Yan Jia, and Xuan Wang. Meta-path based attentional graph learning model for vulnerability detection. *CoRR*, abs/2212.14274, 2022.
- [56] Xin-Cheng Wen, Cuiyun Gao, Feng Luo, Haoyu Wang, Ge Li, and Qing Liao. LIVABLE: exploring long-tailed classification of software vulnerability types. *CoRR*, abs/2306.06935, 2023.
- [57] Alfred V. Aho, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley series in computer science / World student series edition. Addison-Wesley, 1986.
- [58] Robert S. Boyer, Bernard Elspas, and Karl N. Levitt. SELECT - a formal system for testing and debugging programs by symbolic execution. In *Reliable Software*, pages 234–245. ACM, 1975.
- [59] Aniqua Z. Baset and Tamara Denning. Ide plugins for detecting input-validation vulnerabilities. In *2017 IEEE Security and Privacy Workshops (SPW)*, pages 143–146, 2017.
- [60] James Walden and Maureen Doyle. Savi: Static-analysis vulnerability indicator. *IEEE Security & Privacy*, 10(3):32–39, 2012.
- [61] Bill Baloglu. How to find and fix software vulnerabilities with coverity static analysis. In *SecDev*, page 153. IEEE Computer Society, 2016.
- [62] Patrice Godefroid, Michael Y. Levin, and David A. Molnar. Automated whitebox fuzz testing. In *NDSS*. The Internet Society, 2008.
- [63] Vitaly Chipounov, Volodymyr Kuznetsov, and George Candea. S2E: a platform for in-vivo multi-path analysis of software systems. In *ASPLOS*, pages 265–278. ACM, 2011.
- [64] Hongliang Liang, Lei Wang, Dongyang Wu, and Jiuyun Xu. Mlsa: A static bugs analysis tool based on llvm ir. In *2016 17th IEEE/ACIS International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD)*, pages 407–412, 2016.
- [65] Fang Wu, Jigang Wang, Jiqiang Liu, and Wei Wang. Vulnerability detection with deep learning. In *2017 3rd IEEE International Conference on Computer and Communications (ICCC)*, pages 1298–1302, 2017.
- [66] Gustavo Grieco, Guillermo Luis Grinblat, Lucas C. Uzal, Sanjay Rawat, Josselin Feist, and Laurent Mounier. Toward large-scale vulnerability discovery using machine learning. In *CODASPY*, pages 85–96. ACM, 2016.
- [67] Yueming Wu, Deqing Zou, Shihan Dou, Wei Yang, Duo Xu, and Hai Jin. Vulcnn: An image-inspired scalable vulnerability detection system. In *ICSE*, pages 2365–2376. ACM, 2022.
- [68] Huaning Wang, Guixin Ye, Zhanyong Tang, Shin Hwei Tan, Songfang Huang, Dingyi Fang, Yansong Feng, Lizhong Bian, and Zheng Wang. Combining graph-based learning with automated data collection for code vulnerability detection. *IEEE Trans. Inf. Forensics Secur.*, 16:1943–1958, 2021.
- [69] Anh Viet Phan, Minh Le Nguyen, and Lam Thu Bui. Convolutional neural networks over control flow graphs for software defect prediction. In *ICTAI*, pages 45–52. IEEE Computer Society, 2017.
- [70] Xin-Cheng Wen, Yupan Chen, Cuiyun Gao, Hongyu Zhang, Jie M. Zhang, and Qing Liao. Vulnerability detection with graph simplification and enhanced graph representation learning. In *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*, pages 2275–2286. IEEE, 2023.
- [71] Zhen Li, Deqing Zou, Shouhuai Xu, Xinyu Ou, Hai Jin, Sujuan Wang, Zhijun Deng, and Yuyi Zhong. Vuldeepecker: A deep learning-based system for vulnerability detection. In *NDSS*. The Internet Society, 2018.
- [72] Deqing Zou, Sujuan Wang, Shouhuai Xu, Zhen Li, and Hai Jin. *μ*vuldeepecker: A deep learning-based system for multiclass vulnerability detection. *IEEE Trans. Dependable Secur. Comput.*, 18(5):2224–2236, 2021.
- [73] Qian Feng, Rundong Zhou, Chengcheng Xu, Yao Cheng, Brian Testa, and Heng Yin. Scalable graph-based bug search for firmware images. In *CCS*, pages 480–491. ACM, 2016.